Lesson 13　Java で動かす POS レジ

── Web との「違い」を体験しよう

対象: JavaScript / PHP 経験者　**所要時間**: 約 60 分

目標: Java がどんな仕組みで動くかを"雰囲気"として掴む。

※文法の習得は行いません。

Tan（2026/10/15）

タイムスケジュール（60 分）

```
00分〜10分 ▶ 【座学⓪】 Java とは何か？（特徴・言語の違い・完成イメージ）
10分〜20分 ▶ 【座学①】 Web と Java アプリ ── 何が違うの？
20分〜28分 ▶ 【座学②】 Java を動かすための 3 つの道具（JDK/Maven/JavaFX）
28分〜38分 ▶ 【座学③】 プログラムの中身をざっくり見てみよう
38分〜50分 ▶ 【ハンズオン】 ビルド　 起動　 動作確認
50分〜56分 ▶ 【座学④】 ビルドの流れを図で整理する
56分〜60分 ▶ まとめ ＆ Q&A
```

0. Java とは何か？

*** 用語: Java（ジャバ） ***

```
1995年に Sun Microsystems（現在は Oracle に吸収合併済）が開発したプログラミング言語。

世界で最も広く使われている言語の一つで、
Android ・企業の基幹システム・銀行ATM・組み込み機器などの
幅広い分野で採用されている。
```

0-1. Java の立ち位置

Java が使われている身近な例

```
📱 Android アプリ      … スマホアプリの旧開発言語（現在は Kotlin）
🏦 銀行・金融システム … ATM・勘定系システムの多くが Java 製
🏛️ 企業の基幹システム … 在庫管理・販売管理・ERP など大規模業務システム
   デスクトップアプリ  … 今回の JavaFX アプリがこれに該当
☁️ クラウド / Web API … Spring Boot フレームワークで Web サービス
```

0-2. プログラミング言語の「種類」から見た Java の特徴

プログラミング言語はいくつかの「分類」で整理できます。

- インタプリタ言語（逐次実行型）

書いたコードをその場でその場で1行ずつ読みながら動かす

→ 変更がすぐ反映される。実行が始まるまでが早い

例: JavaScript / PHP / Python / Ruby

- コンパイラ言語（事前変換型）

実行前に「全コードを機械語に変換（コンパイル）」してから動かす。

→ 大規模・複雑なシステムに向いている

例: Java / C / C++ / Go / Rust

料理で例えると…

- インタプリタ（JS/PHP）：「レシピを見ながらその場で調理する」

→ すぐ作り始められるけど、途中で失敗してから気づく

- コンパイラ（Java）：「調理前に材料・手順の不備を全部チェックする」

→ 実行前に問題が判明する、大人数分の調理（大人数・大規模システム）に向いている

0-3. Java の 3 大特徴

1. 型がある（静的型付け言語）

JavaScript:

```
let price = 100;      // 型は後から何でも入れられる
price = "100円";      // ← これでも動いてしまう！
```

Java:

```
int price = 100;     //  「整数型」と決めたら
price = "100円";     // ← コンパイル時にエラー！
                     // 型の不一致を実行前に発見できる   |
```

0-3. Java の 3 大特徴

2. どこでも動く（Write Once, Run Anywhere）

Java でビルド（コンパイル）したプログラムは Windows でも Mac でも Linux でも同じファイルで動作する。

→ JVM（仮想マシン）が OS の差を吸収してくれるおかげ

0-3. Java の 3 大特徴

3. オブジェクト指向（クラスとオブジェクト）

JavaScript でもオブジェクト指向でプログラムを書けるが、Java はすべてがオブジェクト単位

プログラム全体が「部品（クラス）の組み合わせ」で構成される

→ 大規模チーム開発・長期保守に強い

0-4. 今回作るもの（完成イメージ）

今日のゴールは、下図のような **デスクトップウィンドウアプリ** を
ビルドして起動する ことです。

（画面イメージ）

▲ ブラウザを使わない！ このウィンドウ自体が Java プログラムの画面
▲ MySQL の売上データをウィンドウ内のテーブルに直接表示している

これまでとの違いを実感しよう

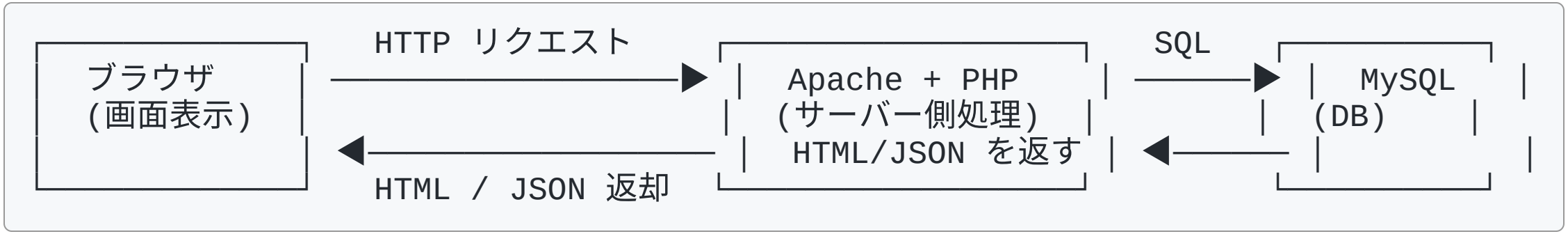
- 今まで: 売上検索 → ブラウザで URL を開いて HTML テーブルを見た
- 今回: 売上検索 → Java ウィンドウを起動して専用画面でテーブルを見る

DB（MySQL）への接続・SQL の送信・結果の画面表示まで、
すべてが *** 1 つの Java プログラム *** の中で完結しています。

1. Web アプリと Java アプリ ── 何が違う？

1-1. これまでの「Web アプリ」の仕組み

これまでの POS システムは、こんな仕組みで動いていました。



▲ 3つの「部品」がネットワーク経由で連携して動いている

復習

*** 用語: HTTP (エイチティーティーピー) ***

ブラウザとサーバーが通信するときの「会話のルール」。

1. 「この URL の HTML をください」
2. 「はい、どうぞ（HTML）」

というやりとりのこと。

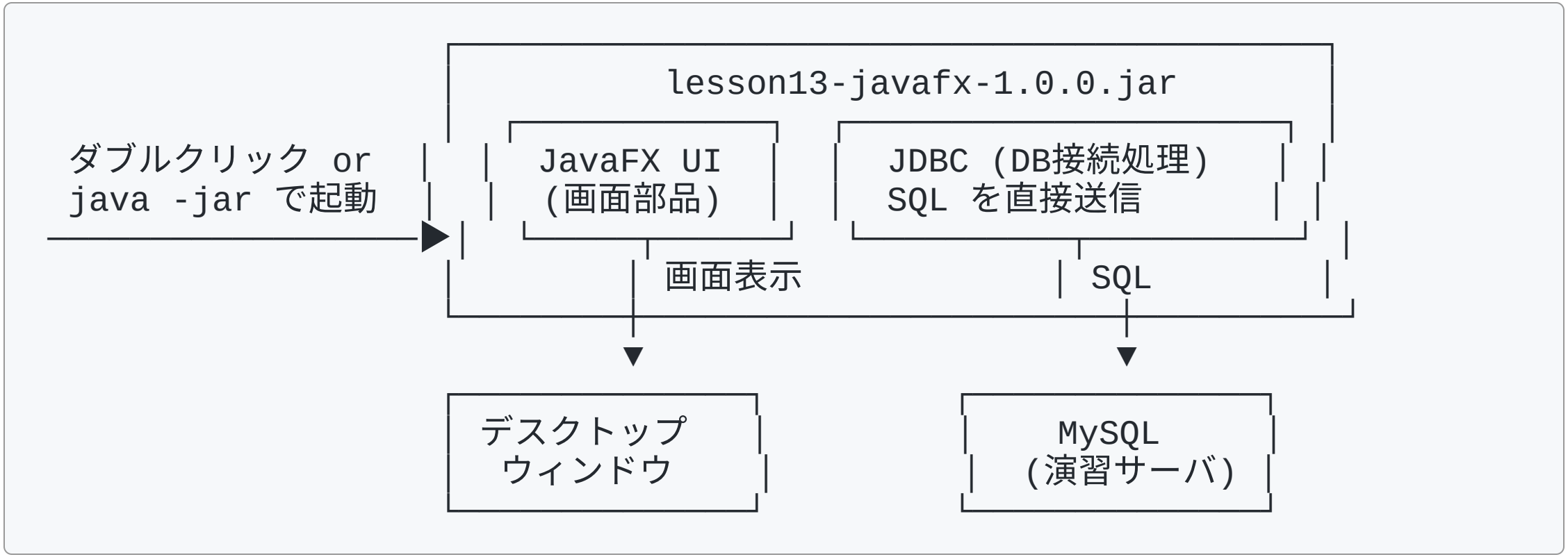
担当ファイルのおさらい:

役割	ファイル	言語
画面の見た目	index.html / CSS	HTML / CSS
画面の動き・通信	main.js	JavaScript
サーバー側処理・DB接続	getProduct.php / db_connect.php	PHP
データ保管	MySQL テーブル	SQL

1-2. 今回の「Java アプリ」の仕組み

Lesson 13 では、**ブラウザも Web サーバーも使いません！**

すべてが **1 つの `.jar` ファイル（Java の実行ファイル）** に詰め込まれています。



▲ ブラウザ不要、Web サーバー（Apache）不要！「画面」も「DB接続」も１ファイルに全部入っている！

*** 💡 ポイント ***

- Web アプリ → ブラウザが「表示係」、Apache が「処理係」と役割が分かれている
- Java アプリ → 1 つのプログラムが「表示も処理も全部やる」

→自己完結型

1-3. Web vs Java ── 考え方の違い 一覧

比較項目	Web アプリ (HTML/PHP)	Java デスクトップアプリ
実行の入口	ブラウザで URL を開く	`java -jar ファイル名.jar` コマンド
画面の作り方	HTML タグ + CSS で記述	JavaFX の「部品クラス」をコードで組み立て
動かすまでの手順	ファイルをサーバーに置くだけ	**コンパイル　ビルド**　実行（3ステップ）
ファイルの配布	.php / .html をコピー	.jar ファイル 1 つをコピー
DB 接続方法	PHP の PDO	Java の JDBC
エラーの確認	ブラウザの開発者ツール	ターミナルのエラーメッセージ

2. Java を動かすための「3 つの道具」

Java でデスクトップアプリを作るには、３種類のソフトウェアが必要です。

料理に例えると…



2-1. JDK（ジェイディーケー）

*** 用語: JDK (Java Development Kit) ***

Java でプログラムを「コンパイルする・実行する」ための開発用キット（道具セット）。

- javac コマンド … ソースコード (.java) を機械語に変換（コンパイル）する
- java コマンド … 変換済みプログラムを実行する

Web 開発での対応	Java での役割
Node.js の実行環境	JDK の実行環境（JRE）
npm の `node` コマンド	`java` コマンド
—	`javac` コマンド（コンパイル。JSには存在しない概念）

2-2. Maven（メイヴン）

*** 用語: Maven ***

Java のプロジェクト管理・ビルド自動化ツール。

`pom.xml` という設定ファイルに「使いたいライブラリ名」を書いておくだけで、インターネットからライブラリを自動ダウンロードして、実行ファイル (.jar) を作ってくれる。

JavaScript の npm との比較:

```
┌──────────────────────────────┬──────────────────────────────┐
│ JavaScript (npm)             │ Java (Maven)                 │
├──────────────────────────────┼──────────────────────────────┤
│ package.json                 │ pom.xml                      │
│   "dependencies": {          │   <dependencies>             │
│     "axios": "^1.0"          │     <artifactId>             │
│   }                          │       javafx-controls        │
│                              │     </artifactId>            │
│ npm install  → 自動DL        │ mvn package → 自動DL         │
│ node_modules/ に保存         │ ~/.m2/ に保存                │
└──────────────────────────────┴──────────────────────────────┘
```

▲ 考え方はほぼ同じ！ npm　Maven

2-3. JavaFX SDK（ジャバエフエックス）

*** 用語: JavaFX SDK ***

Java でデスクトップ画面（GUI）を作るための「部品ライブラリ」。

ボタン・テキスト入力欄・テーブル表示など、画面パーツが揃っている。

今回は Maven が `pom.xml` の設定を読んで自動で取り込むため、手動でインストールする必要はない。

HTML タグとの対応：

HTML タグ	JavaFX クラス	役割
`<button>`	`Button`	クリックできるボタン
`<input type="text">`	`TextField`	1行テキスト入力欄
`<table>`	`TableView`	グリッド表示のテーブル
`<span>` / `<p>`	`Label`	文字を表示するラベル

HTML タグとの対応：

HTML タグ	JavaFX クラス	役割
`<div style="display:flex; flex-direction:column">`	`VBox`	縦並びレイアウト
`<div style="display:flex; flex-direction:row">`	`HBox`	横並びレイアウト
`alert("エラー")`	`Alert`	エラーポップアップ

3. プログラムの中身をざっくり見てみよう

`l13_Pos_regi.java` を開いてみましょう。全 438 行ありますが、

構造は 4 つのブロックに分けると理解しやすいです。

3-1. ファイルの全体構造（I13_Pos_regi.java の構造）

```
① package / import 宣言（行 1〜230 ）
   ├─ package com.example.lesson13;   ← このファイルの「住所」宣言
   └─ import javafx.xxx ...           ← 使うライブラリを事前宣言
      import java.sql.*;

② クラス定義ヘッダー（行 231〜280 ）
   ├─ DB 接続情報（ホスト名・ユーザー名・パスワード）の定数
   └─ 画面部品の変数宣言（TextField・Button・Label・TableView）

③ start() メソッド   ← 画面を組み立てて表示する
   │  JavaFX が「最初に自動呼び出し」する関数
   │  JS の window.onload や DOMContentLoaded に相当
   │                                        （行 281〜350 ）
   └─ 画面部品を VBox / HBox に配置して Stage に載せる

④ performSearch() メソッド   ← 検索ボタンを押したときの処理
   │  ① MySQL に JDBC で接続
   │  ② SQL を PreparedStatement で安全に実行
   │  ③ 結果を ResultSet で受け取り TableView に表示
   └                                        （行 351〜460 ）

⑤ SalesDetailRow クラス（内部クラス）
   └─ テーブルの1行分のデータを保持する「箱」（行 461〜末尾）
```

3-2. ① package と import ── JavaScript との比較

```
// ── ① このファイルの「住所」宣言 ──────────────────────────────
package com.example.lesson13;
//  JS でいう: src/example/lesson13/ フォルダに置いてある、というイメージ

// ── ② 使う「部品」を予め宣言する ──────────────────────────────
import javafx.scene.control.*;  // Button / Label / TextField / TableView 等
import javafx.scene.layout.*;   // VBox（縦並び）/ HBox（横並び）
import javafx.stage.Stage;      // ウィンドウ（Window）そのもの
import java.sql.*;              // DB接続 (= PHP の PDO に相当)
```

** 💡 <b>JavaScript との比較ポイント **

JavaScript:

```
import { Button } from 'javafx-controls';
```

Java:

```
import javafx.scene.control.Button;
```

「使いたいものは先に宣言する」という考え方は JavaScript の import と全く同じ！

Java では **クラス名（部品名）まで正確に書く** ルールがある。
`*` （アスタリスク）で「このフォルダの全部品を一括取込」も可能。

3-3. ③ start() ── 画面の組み立て

HTML/JavaScript/PHP では HTML タグをテキストで書きますが、Java では **コードで部品を作って並べます。**

HTML の書き方	Java (JavaFX) の書き方

`<div style="display:flex;`
`  flex-direction:column">`
`  <h1>タイトル</h1>`
`  <input type="text">`
`  <button>検索</button>`
`  <table>...</table>`
`</div>`
	`VBox mainLayout = new VBox();`
	`Label title =`
	`  new Label("タイトル");`
	`TextField input =`
	`  new TextField();`
	`Button btn =`
	`  new Button("検索");`
	`TableView table =`
	`  new TableView();`
	`mainLayout.getChildren()`
	`  .addAll(title,input,btn,`
	`  table);`

↑ テキストで構造を「書く」	↑ コードで部品を「組み立てる」

ボタンのクリックイベントの比較：

```
JavaScript:
  document.getElementById('btn').addEventListener('click', performSearch);

Java:
  searchButton.setOnAction(e -> performSearch());
```

** 💡 <b>ポイント **

`e -> performSearch()` は JS の arrow 関数 `(e) => performSearch()` と全く同じ考え方！

「ボタンが押されたら `performSearch()` を呼ぶ」という意味です。

3-4. ④ performSearch() ── DB 検索の流れ

performSearch() の処理の流れ

① 入力欄から売上番号を取得（文字列）
↓
② 数字かどうかチェック（数字でなければエラーポップアップ）
↓
③ MySQL に JDBC 接続

```
PHP の書き方:
  $pdo = new PDO("mysql:host=10.8.0.1", $u, $p);

Java の書き方:
  Connection conn =
    DriverManager.getConnection(url, user, pass);
```

↓
④ SQL を安全に実行（売上ヘッダー情報の取得）

```
PHP:  $stmt = $pdo->prepare("SELECT ... WHERE
        tsh_sales_no = ?");
       $stmt->execute([$salesNo]);

Java: PreparedStatement ps =
        conn.prepareStatement("SELECT ...");
       ps.setInt(1, salesNo);
       ResultSet rs = ps.executeQuery();
```

↓
⑤ 結果を1行ずつ読み取り → TableView に追加

```
PHP:  while ($row = $stmt->fetch()) {
        $row['mcu_name'];
       }

Java: while (rs.next()) {
        rs.getString("mcu_name");
       }
```

↓
⑥ 画面（TableView）が自動的に更新される

*** 💡 <b>SQL 文自体は Python とまったく同じ！ ***

`SELECT` / `LEFT JOIN` / `WHERE` / `ORDER BY` の書き方はそのまま使えます。

変わるのは「SQL を送り込む方法（PDO → JDBC）」だけです。

4. 【ハンズオン】ビルド　起動　動作確認

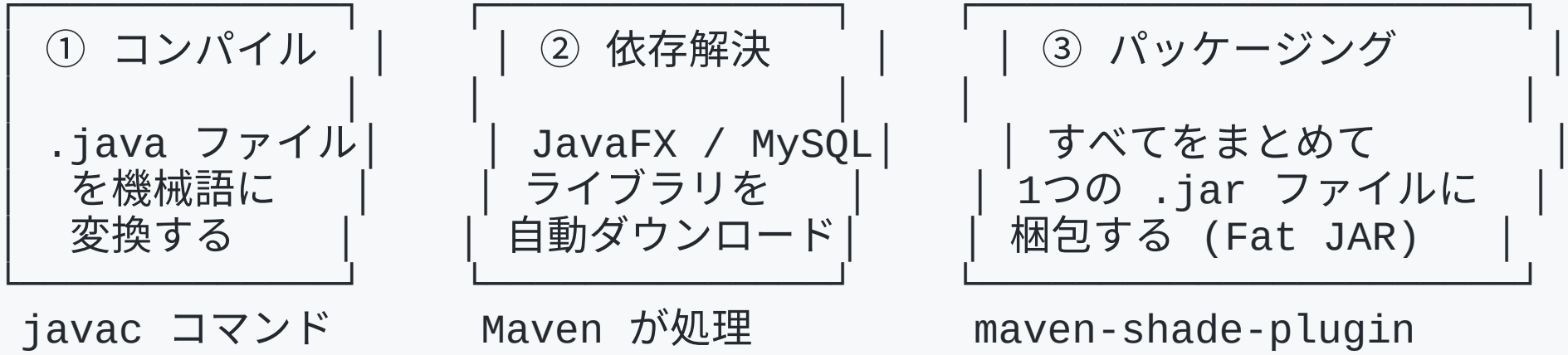
4-1. ビルド（コンパイル）とは何か？

*** 用語: ビルド (Build) ***

「ソースコード (.java ファイル) を実行できる形 (.jar ファイル) に変換する」一連の作業のこと。

料理でいえば「材料を集めて → 調理して → 完成品（弁当）に詰める」全工程。

ビルドの 3 ステップ（Maven が全自動でやってくれる！）



*** 用語: Fat JAR (ファット・ジャー) ***

必要なライブラリをすべて 1 つのファイルに詰め込んだ実行ファイル（.jar）のこと。

「太った（Fat）＝中身が詰まった jar」という意味。

配布が **1 ファイルをコピーするだけ** で完了するメリットがある。

4-2. ビルド手順

コマンドプロンプトを開き、以下を実行します。

ステップ 1: lesson13 フォルダに移動

```
cd C:\Users\m-baba\MyData\2026年度_IT人材\2.基礎編\プログラム\2026\user99\lesson13
```

ステップ 2: ビルドを実行

```
build.bat
```

`build.bat` を実行すると内部で `mvn clean package` が呼ばれ、 Maven が自動的に以下を行います：

build.bat 実行中の内部動作（目安: 1〜3 分）

```
  [1] Maven: pom.xml を読み込む
  [2] Maven: JavaFX (21.0.2) ライブラリをインターネットから取得
  [3] Maven: MySQL ドライバ (8.3.0) をインターネットから取得
  [4] javac: l13_Pos_regi.java と Launcher.java をコンパイル
  [5] Maven: すべてを lesson13-javafx-1.0.0.jar に梱包
```

✅ 成功すると最後に以下が表示されます：

```
[INFO] BUILD SUCCESS
```

*** ⚠️ <b>初回ビルドは時間がかかります ***

ライブラリのダウンロードに数分かかることがあります。

2回目以降は PC に保存済みのため高速（数秒）で完了します。

4-3. アプリケーションを起動する

ビルド成功後、以下のコマンドで起動します：

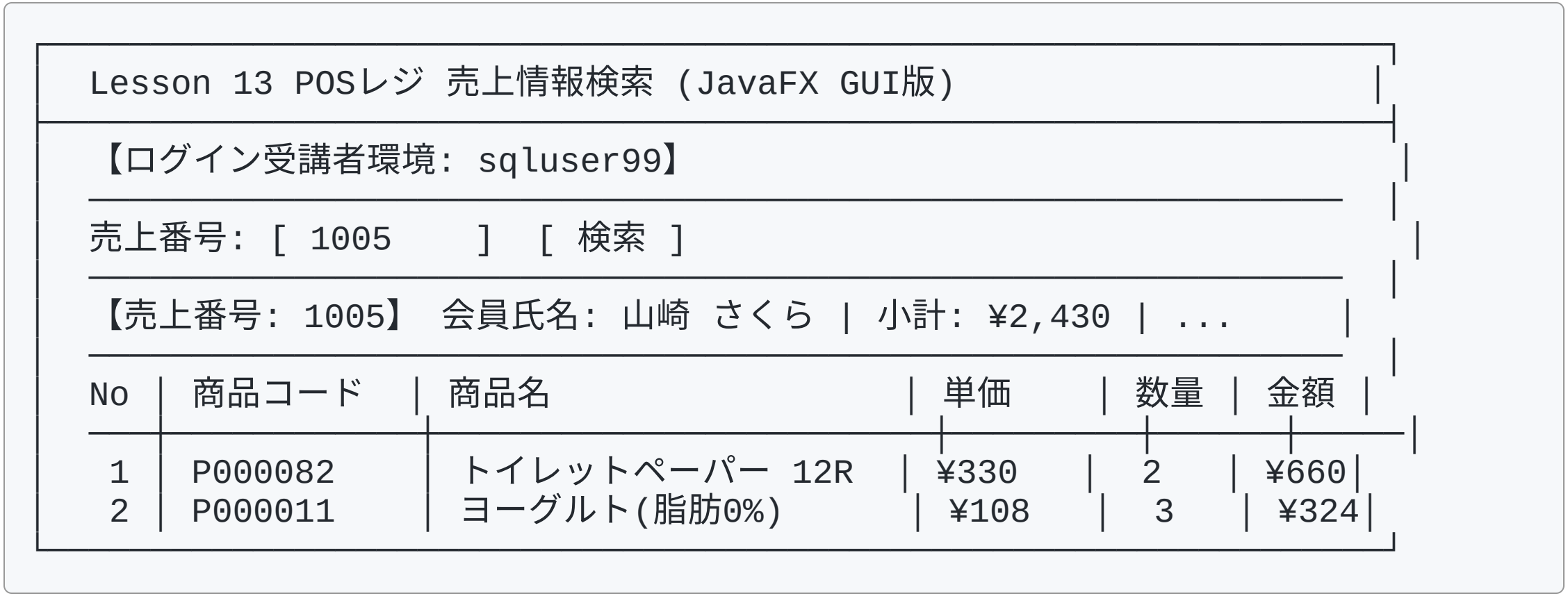
```
java --enable-native-access=ALL-UNNAMED -jar target\lesson13-javafx-1.0.0.jar
```

*** 用語: --enable-native-access=ALL-UNNAMED ***

Java 21 以降のセキュリティ設定で、デスクトップ画面（JavaFX）が
Windows のネイティブ機能（ウィンドウ描画）を使用する際に必要なオプション。

おまじないとして覚えておけば OK！

起動すると、**デスクトップにウィンドウが表示されます。**



4-4. 動作確認

売上番号に `1005` と入力して「検索」ボタンをクリックしてみましょう。

確認項目	期待する結果
ヘッダー部分	会員名・小計・消費税・合計金額が表示される
明細テーブル	商品コード・商品名・単価・数量・金額が一覧表示される
存在しない番号を入力	「該当する売上情報が見つかりません」が赤表示される
数字以外を入力	エラーポップアップが表示される

5. ビルドの全体像を整理する

5-1. Web アプリ vs Java アプリ ── 動かすまでの流れ比較

【Web アプリ（Python）を動かすまで】

① .py / .html ファイルを作成
↓
② Apache サーバーの所定フォルダに置く
↓
③ ブラウザで URL を開く → 即実行！

✅ 手順が少なく、変更がすぐ反映される
❌ Web サーバーが必要、配布が複数ファイルになる

【Java アプリ（JavaFX）を動かすまで】

① .java ファイルを作成（今回は作成済み）
　　↓
② コンパイル: .java → .class（機械語に変換）
　　↓　　Maven が自動でやってくれる
③ パッケージング: .class + ライブラリ → .jar（Fat JAR）
　　↓
④ java -jar ファイル名.jar で実行！

✅ 1ファイル（.jar）で配布・実行できる
✅ Web サーバー不要
❌ 変更のたびにビルドが必要

5-2. ファイルの役割マップ

lesson13/ フォルダ構成と各ファイルの役割

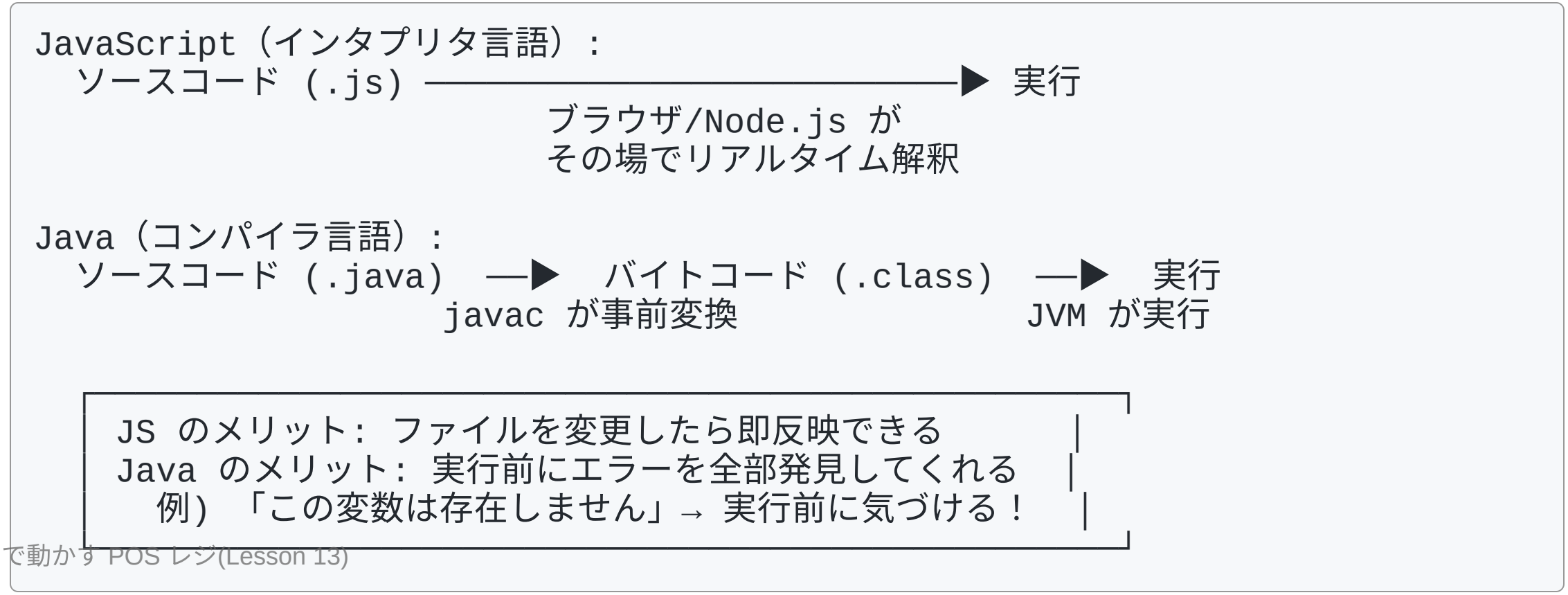
```
  lesson13/
  │
  ├── pom.xml                ← 📋 Maven の設定書
  │                             「JavaFX と MySQL を使う」と書いてある
  │                             JS の package.json に相当
  │
  ├── l13_Pos_regi.java      ← 💻 メインプログラム
  │                             画面の構築 + JDBC による DB 検索処理
  │
  ├── Launcher.java          ← 🚪 Fat JAR 起動の入り口
  │                             「java -jar」したときに最初に呼ばれる
  │                             JS の index.js (エントリーポイント) に相当
  │
  ├── build.bat              ← 🔨 ビルド実行バッチ
  │                             「mvn clean package」を呼ぶだけのスクリプト
  │
  └── target/
      └── lesson13-javafx-1.0.0.jar  ← 📦 完成品（Fat JAR）
                                        これ 1 つで動く実行ファイル！
```

5-3. コンパイルとは何か？（JS との最大の違い）

*** 用語: コンパイル (Compile) ***

人間が書いたソースコードを、コンピュータが直接実行できる形式（機械語・バイトコード）に「事前変換」する作業のこと。

この変換作業を行うプログラムを「コンパイラ」と呼ぶ。



*** 用語: JVM (Java Virtual Machine) ***

Java のバイトコード (.class) を実行する「仮想マシン」。

Windows / Mac / Linux など OS の違いを吸収し、 *** どの OS でも同じ .jar ファイルが動く *** のはこの JVM のおかげ。

（"Write once, run anywhere" ── １回書けばどこでも動く）

6. まとめ ── 今日覚えてほしいこと

6-1. 今日のポイント 3 つ

```
┌──────────────────────────────────────────────┐
│ ① Java アプリは「ビルド」してから動かす      │
│    .java → (コンパイル) → .class → (梱包) → .jar │
│    Web のように「ファイルを置けばすぐ動く」ではない │
├──────────────────────────────────────────────┤
│ ② 考え方は JS/Python と同じ、書き方だけが違う │
│    イベント処理・DB接続・SQL文の考え方はそのまま使える！ │
│    「部品の組み立て方」が HTML タグではなくコードになる │
├──────────────────────────────────────────────┤
│ ③ Maven が「npm」の役割をしてくれる          │
│    pom.xml に書くだけでライブラリを自動取得  │
│    build.bat 1 本で「コンパイル + 梱包」まで全自動 │
└──────────────────────────────────────────────┘
```

6-4. 【補足】今後 Java をもっと学ぶなら

本レッスンは「Java を体験する」ことが目的でした。

もし今後 Java を本格的に学びたい場合は、以下のキーワードで調べてみてください：

- **オブジェクト指向（OOP）** ── `class` / `extends` / `interface` の考え方
- **型システム** ── `int` / `String` / `boolean` など静的型付け
- **例外処理（Exception）** ── `try / catch / finally` の詳細な使い方
- **Stream API** ── Java でのリスト・配列の高度な処理（JS の `map` / `filter` に相当）

Lesson 13 以上